

Tech Leaders · Class 16

Motion-Controlled Game

Build the controller AND the game it plays

Joystick axes

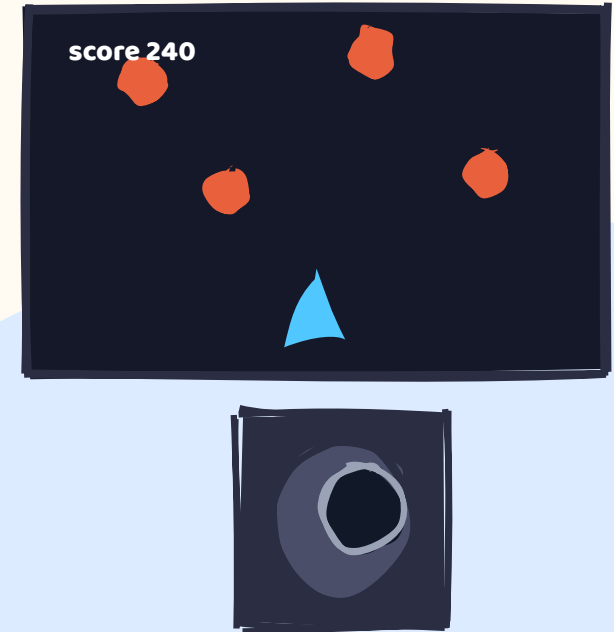
Serial protocol

Pygame

Feedback

Did you know?

The Xbox Adaptive Controller was designed with gamers with disabilities, with big pads and ports for custom switches.





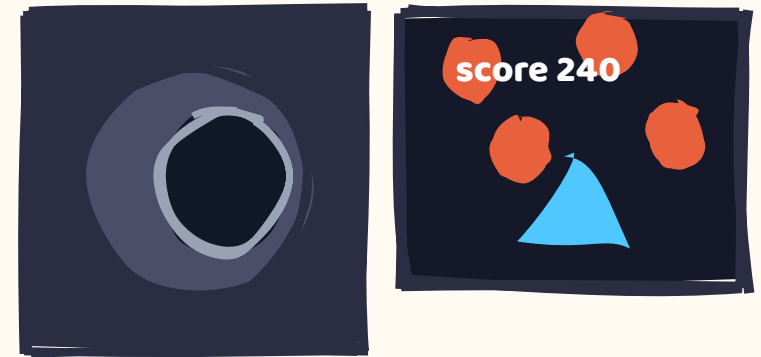
Let's think...

What happens inside a controller in 1/60 s?



Push the stick...

You push a joystick and the character moves **instantly**. In that moment, the controller measured your thumb, turned it into numbers, sent them to the console, and the game drew a new frame. If any step is slow, the game feels 'laggy'. Let's build every step ourselves.



Magic wires?

Tiny cameras?

Numbers, fast!

Today: you build a joystick controller with Arduino and a **Pygame** dodging game it controls, with feedback both ways.





Learning objectives

By the end of this class I can...



1

**Joystick:** read a joystick as **two analog axes** and add a **dead zone****Proof:** Serial shows 0 when the stick is centred, ± 1 at the edges

2

**Protocol:** design a simple **serial protocol** between Arduino and laptop**Proof:** our protocol written down and working at 50 messages a second

3

**Game loop:** build a **Pygame** game whose player is moved by our controller**Proof:** the ship dodges rocks at a steady 60 FPS

4

**Feedback:** send **events back** to the controller for light and sound**Proof:** the buzzer and LED react when the ship is hit

5

**Playtest:** playtest for fun and **accessibility**, then improve**Proof:** two changes made from feedback



Objective 1 • Joystick • video

Video time!



How the Wii Remote Works

VonTower • about 4 minutes • youtu.be/ETAKfSkec6A

Watch for...

- 1 Which **sensors** are inside the Wii Remote?
- 2 How does the console know where you are **pointing**?
- 3 How does the controller send data to the console?



After the video: Our controller uses a joystick. Which Wii Remote sensor would you add to make it truly 'motion' controlled?





Objective 1 • Joystick

A joystick is two knobs at right angles



The idea

Inside are two **potentiometers**: one for **X** (left-right), one for **Y** (up-down). Each reads **0 to 1023**, about **512** at rest.

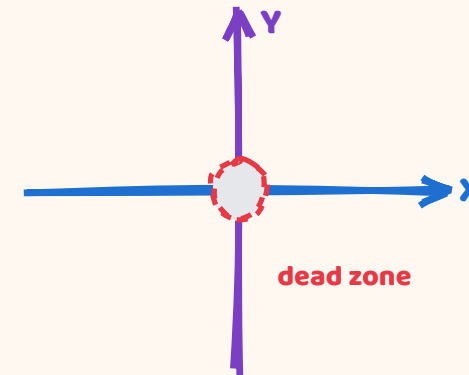
Convert to **-1 ... +1**: $(\text{value} - 512) \div 512$.

Real sticks never rest at exactly 512, so we add a **dead zone**: anything within ± 60 counts as 0. No more 'drifting' ship!

Hidden button

Pressing the stick down is a button too (SW). It reads LOW when pressed.

Example: a car steering wheel



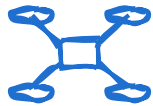
A steering wheel has a little 'play' in the middle: tiny wobbles don't turn the car. That play is a dead zone. Without it, the car would weave every time your hands trembled.





Real world

Controllers beyond games



Drones

Two sticks: altitude, rotation and direction



Robot arms

Joysticks guide surgical and bomb-disposal robots



Wheelchairs

Powered wheelchairs steer with a joystick



Adaptive gaming

Custom switches let everyone play



A powered wheelchair joystick: why is a dead zone a SAFETY feature, not just a nicety?

Precision vs safety

Medical controls use bigger dead zones and speed limits than games.



Class game

Human game loop

How to play

1. Three students: **Controller**, **Cable**, **Game**.
2. Controller shows a card every 2 s: '**x=700, y=512**'.
3. Cable runs it to the Game, who moves a paper ship on a grid and calls out 'hit!' if it touches a rock.
4. Cable runs 'hit' back: Controller buzzes (says BZZZ).
5. Now speed up to every 1 s. What breaks?

Then discuss

Where was the delay? What if the Game waited for every message before drawing the next frame?

Big idea

Fast, small, regular messages and a game that never waits: that's how real controllers feel smooth.



Build it

Our controller kit (per group)

**Arduino Uno**

Reads the controls

**Joystick module**

X, Y and a click button

**2 push buttons + 10k**

Fire and pause

**LED + 220 Ω**

Flashes when hit

**Buzzer**

Sound feedback

**Laptop**

Python + Pygame



Install once: **pip install pygame pyserial**. Put the parts in a cardboard box with holes: now it feels like a real controller.

Ergonomics

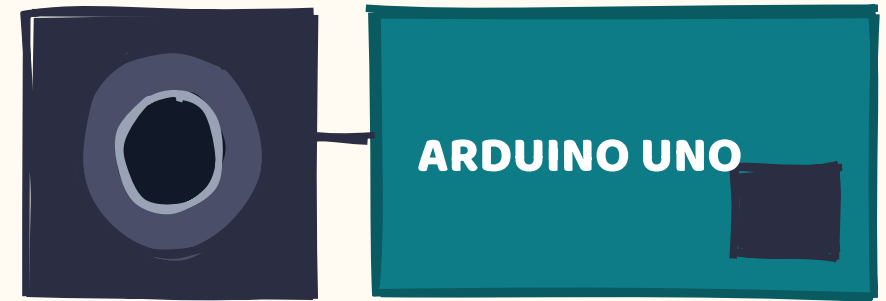
Hold your box for 2 minutes. Where do your thumbs naturally rest? Put the stick and buttons there.



Objective 1 • Joystick • Build step 1 of 4

Wire the controller

Part	Connect to
Joystick VRx / VRy	A0 / A1
Joystick SW	pin 2 (INPUT_PULLUP)
Joystick +5V / GND	5V / GND
Button A, B (10k to GND)	pins 3, 4 and 5V
LED + 220 Ω	pin 12
Buzzer +	pin 11



Why 10k?

Without the pull-down resistor, the button pin 'floats' and fires randomly. SW uses the Arduino's built-in pull-up instead.



Objective 2 · Protocol

Design a protocol: agree the message



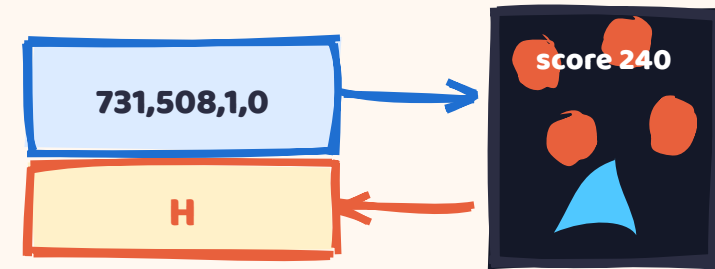
The idea

A **protocol** is an agreement about messages. Ours:

- Arduino → laptop, every 20 ms: **x,y,a,b** then newline, e.g.
731,508,1,0
- Laptop → Arduino, one letter: **H** = hit, **P** = point, **O** = game over

Small text messages are easy to debug in the Serial Monitor.
50 messages a second is plenty for smooth control.

Example: radio call signs



Pilots and air traffic control use strict short phrases: 'Roger', 'Wilco', 'Mayday'. Both sides know exactly what each word means, so messages are fast and never confused.





Objective 2 · Protocol · Build step 2 of 4

Arduino: send controls, receive events



```
const int VX = A0, VY = A1, SW = 2, BTN_A = 3, BTN_B = 4;
const int LED = 12, BUZ = 11;

void setup() {
  Serial.begin(115200);
  pinMode(SW, INPUT_PULLUP);
  pinMode(BTN_A, INPUT); pinMode(BTN_B, INPUT);
  pinMode(LED, OUTPUT);
}

void loop() {
  Serial.print(analogRead(VX));      Serial.print(',');
  Serial.print(analogRead(VY));      Serial.print(',');
  Serial.print(digitalRead(BTN_A));  Serial.print(',');
  Serial.println(digitalRead(SW) == LOW); // stick click

  if (Serial.available()) {          // messages from the game
    char c = Serial.read();
    if (c == 'H') { digitalWrite(LED, HIGH); tone(BUZ, 200, 200); }
    if (c == 'P') { tone(BUZ, 1200, 40); }
    if (c == 'O') { digitalWrite(LED, LOW); }
  }
  delay(20);                          // about 50 messages per second
}
```

Test first

Open Serial Monitor at 115200. Move the stick: do the first two numbers change smoothly?

Two-way

The same USB cable carries messages **both ways** at once.

Close it!

Close the Serial Monitor before running Python. Only one program can use the port.





Objective 3 • Game loop

The game loop must never wait



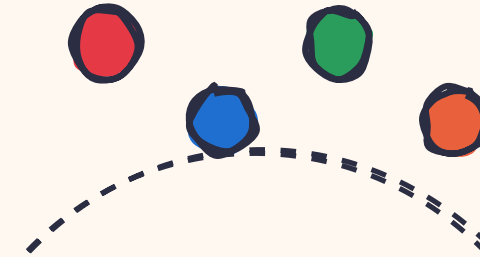
The idea

Every frame (60 times a second):

1. **Read input:** take whatever serial bytes have arrived, without waiting.
2. **Update:** move the ship, move the rocks, check collisions.
3. **Draw** the frame and **tick** the clock to keep 60 FPS.

timeout=0 on the serial port means 'give me what you have now'. If the game waited for each message, the whole screen would stutter.

Example: a juggler



A juggler doesn't stop to wait for one ball. They glance, catch what's arriving, throw, and keep the rhythm going. A game loop does the same with input, updates and drawing.



Objective 3 • Game loop • Build step 3 of 4

Pygame part 1: set up and read

```

import pygame, serial, random

ser = serial.Serial("COM5", 115200, timeout=0) # timeout=0: never wait
pygame.init()
screen = pygame.display.set_mode((800, 600))
clock = pygame.time.Clock()
font = pygame.font.SysFont(None, 36)

ship = pygame.Rect(380, 520, 40, 40)
rocks, score, buf = [], 0, ""
jx = jy = 0.0
DEAD = 60

def axis(v):
    # 0..1023 -> -1..+1 with dead zone
    v -= 512
    return 0.0 if abs(v) < DEAD else v / 512

def read_controller():
    global buf, jx, jy
    buf += ser.read(ser.in_waiting).decode(errors="ignore")
    while "\n" in buf: # handle every full line
        line, buf = buf.split("\n", 1)
        p = line.strip().split(",")
        if len(p) == 4 and all(s.isdigit() for s in p):
            jx, jy = axis(int(p[0])), axis(int(p[1]))

```

Buffer

Bytes arrive in random-sized chunks. We collect them in **buf** and only use **complete lines**.

Validate

isdigit() throws away half-received or garbled lines instead of crashing.

Port name

Windows: COM3, COM5... Mac: /dev/cu.usbmodem... Check the Arduino IDE's Tools → Port.



Objective 4 • Feedback • Build step 4 of 4

Pygame part 2: the game loop



```

running = True
while running:
    for e in pygame.event.get():
        if e.type == pygame.QUIT:
            running = False

    read_controller()                # 1. input
    ship.x += int(jx * 9)            # 2. update
    ship.y += int(jy * 9)
    ship.clamp_ip(screen.get_rect())
    if random.random() < 0.04:
        rocks.append(pygame.Rect(random.randint(0, 770), -30, 30, 30))
    for r in rocks:
        r.y += 6
    if any(ship.colliderect(r) for r in rocks):
        ser.write(b"H")              # feedback!
        rocks.clear(); score = 0
    rocks = [r for r in rocks if r.y < 600]
    score += 1

    screen.fill((20, 24, 40))        # 3. draw
    pygame.draw.rect(screen, (80, 200, 255), ship)
    for r in rocks:
        pygame.draw.rect(screen, (240, 120, 60), r)
    screen.blit(font.render(f"score {score}", True, (255, 255, 255)), (10, 10))
    pygame.display.flip()
    clock.tick(60)                   # 60 FPS
pygame.quit()

```

ser.write(b"H")

The game tells the controller what happened. The buzzer and LED make the hit **felt**, not just seen.

clamp_ip

Keeps the ship inside the screen.

Y upside down?

If pushing up moves the ship down, use **ship.y -= ...** or swap the sign in `axis()`.





Objective 4 • Feedback

Feedback makes it feel real



The idea

Players feel a game through **three channels**: what they see, hear and feel. Our controller adds:

- **Sound**: low buzz on a hit, short beep per 100 points (send 'P')
- **Light**: LED on when hit, off on restart

Real controllers use **vibration motors** (haptics). Feedback also helps players who can't see the screen well.

Example: a phone keyboard



Many phones give a tiny vibration or click on each key press. You type faster and more accurately because your fingers 'feel' each press. Turn it off and typing feels strangely dead.





Objective 5 • Playtest

Playtest for fun AND for everyone



Question	Answer
Fun (1–5)	
Too easy / too hard? When?	
Controller comfortable after 2 min?	
Playable with ONE hand?	
Playable with sound only?	
Our change after feedback	

Accessibility ideas

- Bigger dead zone option for shaky hands
- Buttons that can be remapped
- Slower 'relaxed' mode
- Different sounds for left and right rocks



Watch, don't explain

Give the controller to a tester and stay silent. Where they struggle is your to-do list.





Test like an engineer

Debug and measure



Check	✓ / ✗	How
Serial shows x,y,a,b lines		
Centred stick = ship still		
Game runs at 60 FPS		<code>print(clock.get_fps())</code>
Hit → LED + buzz		
Messages per second		count lines for 1 s

Debug checklist

Ship drifts? Increase DEAD.
 'Access denied'? Serial Monitor still open.
 Game laggy? Remove `print()` calls inside the loop.



Measure, don't guess: print the FPS and the number of messages per second.





Level up!

Game designer challenges



Choose your challenge



Fire!: button A shoots bullets that destroy rocks (+10 points, send 'P').



True motion: add an MPU6050 and steer by tilting the box.



Difficulty curve: rocks speed up every 500 points.



Two players: a second joystick on A2/A3 controls a second ship.

Think like an engineer

Why do most games use a curve (like x^3) instead of a straight line for joystick speed?



Boss level

Measure your controller's lag: flash the LED and the screen together, film in slow motion, count frames.





Objective check

Can I really do it? Prove it.



01 Show the joystick values at rest and at the edges. What does the dead zone do?



02 Write our protocol on the board: what goes each way, and how often?



03 Run the game. What are the 3 steps of the game loop, and why timeout=0?



04 Get hit. What happens on the controller, and which line of code caused it?



05 Read your playtest notes. What did you change for accessibility?



Rate yourself

Colour one circle each. Red: not yet. Amber: with help. Green: I can explain it to a friend.

Teacher: Arcade hour: every team's game on a table. Visitors play and leave sticky-note feedback.





Phase 1 complete!

Tech Leaders showcase



You built 16 systems

Solar weather node · maze robot · robotic arm · fall detector · ESP-NOW network · cloud smart home · parking dashboard · two-factor locker · data pipeline · ML rain predictor · fake news detector · REST API · IoT security · fire-fighting robot · parametric gripper · motion game

Showcase idea

Pick your favourite project, improve it, and present it as a **product**: problem, demo, data, limitations, next version.



Capstone ideas

Autonomous delivery robot · full-stack IoT product with dashboard · predictive plant health analytics · assistive device for a person with a disability



Show what you know

What did we learn?



- ☒ A joystick is two potentiometers; a dead zone stops drift
- ☒ A protocol is an agreement: small, regular, easy-to-debug messages
- ☒ The game loop reads, updates and draws, and never waits
- ☒ Feedback through sound, light and touch makes games feel real
- ☒ Playtest with real people, including for accessibility



Exit ticket: the ship slowly drifts left when nobody touches the stick. Why, and what is the fix?



At home

Watch someone play a game at home. Note one moment the controller gave feedback (sound, vibration, light).



Next class:
Phase 2: Vision & Intelligence!

